

Socket TCP

- Indirizzi

in netinet/ip.h:

```
struct sockaddr_in {  
    sa_family_t    sin_family;  
    u_int16_t      sin_port;  
    struct in_addr sin_addr;  
};
```

```
struct in_addr {  
    u_int32_t      s_addr;  
};
```



AF_INET

porta, in network order

indirizzo IP, in network order

vedere: man 7 ip

Socket TCP

- Indirizzi

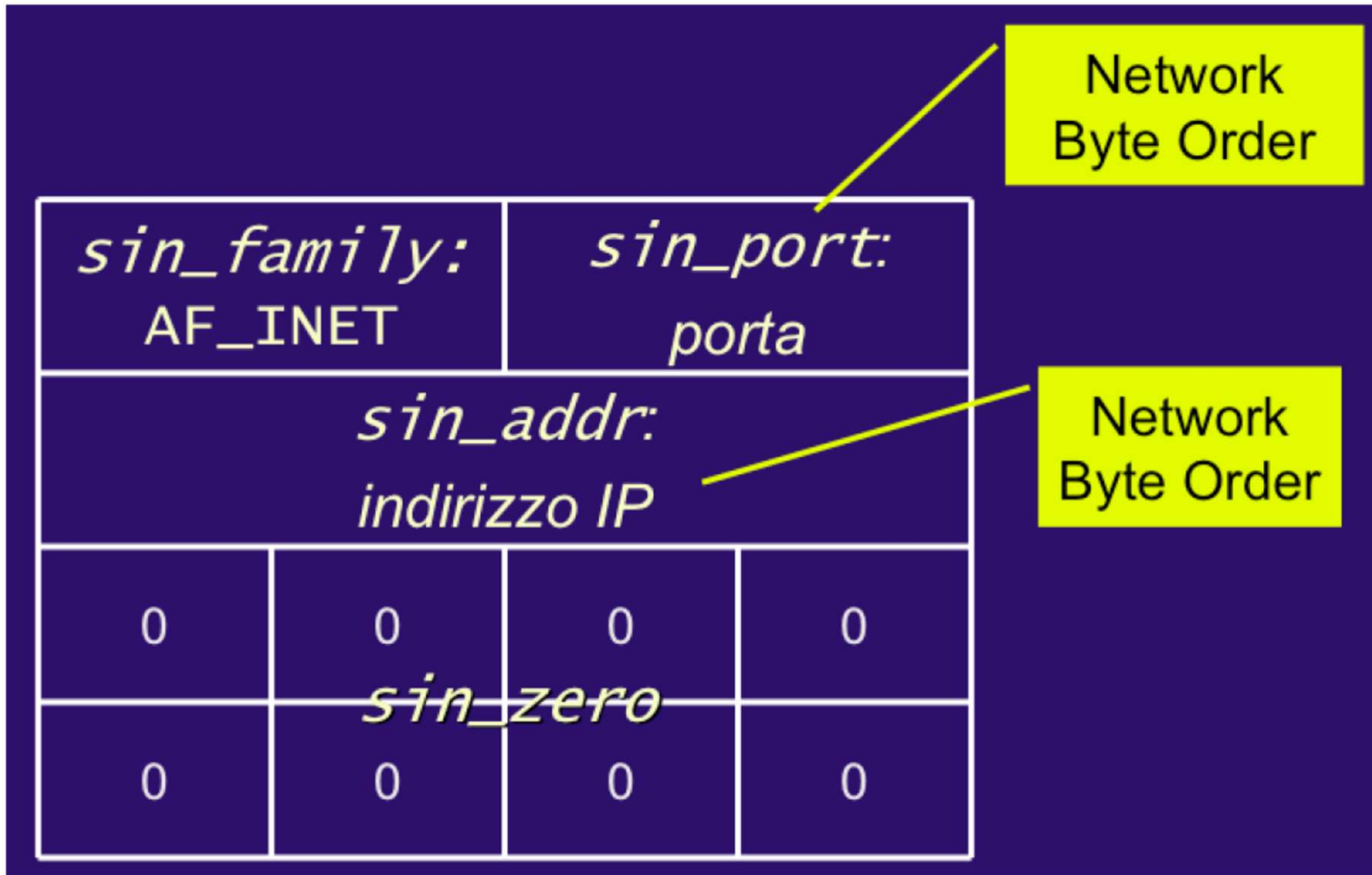
- Siamo interessati a `sa_family = AF_INET`, cioè al dominio “internet”
- In tal caso servono 2 byte per un numero di porta e 4 byte per un indirizzo IP

```
#include <netinet/in.h>
struct sockaddr_in {
    short int sin_family;           // Address family
    { unsigned short int sin_port;  // Port number
      struct in_addr sin_addr;     // IP Internet Address
      unsigned char sin_zero[8];   // Stessi byte di...
    };                             // ...struct sockaddr
```

→ 14 bytes

Socket TCP

- Indirizzi



Socket TCP

- Indirizzi

- Identificati da:

- 1) un indirizzo IP (intero a 32 bit, es. 143.225.5.3)
- 2) una porta (intero a 16 bit, da 0 a 65535)

- Porte:

- per offrire diversi servizi dallo stesso indirizzo IP
- da 0 a 1023: porte riservate (ai processi di root)
- da 5000 a 32768: porte utente
- altre: porte effimere (per i client, ai quali non interessa scegliere una porta specifica)

Socket TCP

- Indirizzi
 - Identificati da:
 - 1) un indirizzo IP (intero a 32 bit, es. 143.225.5.3)
 - 2) una porta (intero a 16 bit, da 0 a 65535)
 - Porte:
 - per offrire diversi servizi dallo stesso indirizzo IP
 - da 0 a 1023: porte riservate (ai processi di root)
 - da 5000 a 32768: porte utente
 - altre: porte effimere (per i client, ai quali non interessa scegliere una porta specifica)

Socket TCP

- Esempi di porte riservate

- 21 ftp (trasferimento file)
- 22 ssh (login remoto sicuro)
- 25 smtp (invio email)
- 79 finger (informazioni sugli utenti)
- 80 http (web)
- 143 imap (lettura email)

- Lista ufficiale su: <http://www.iana.org/>

Socket TCP

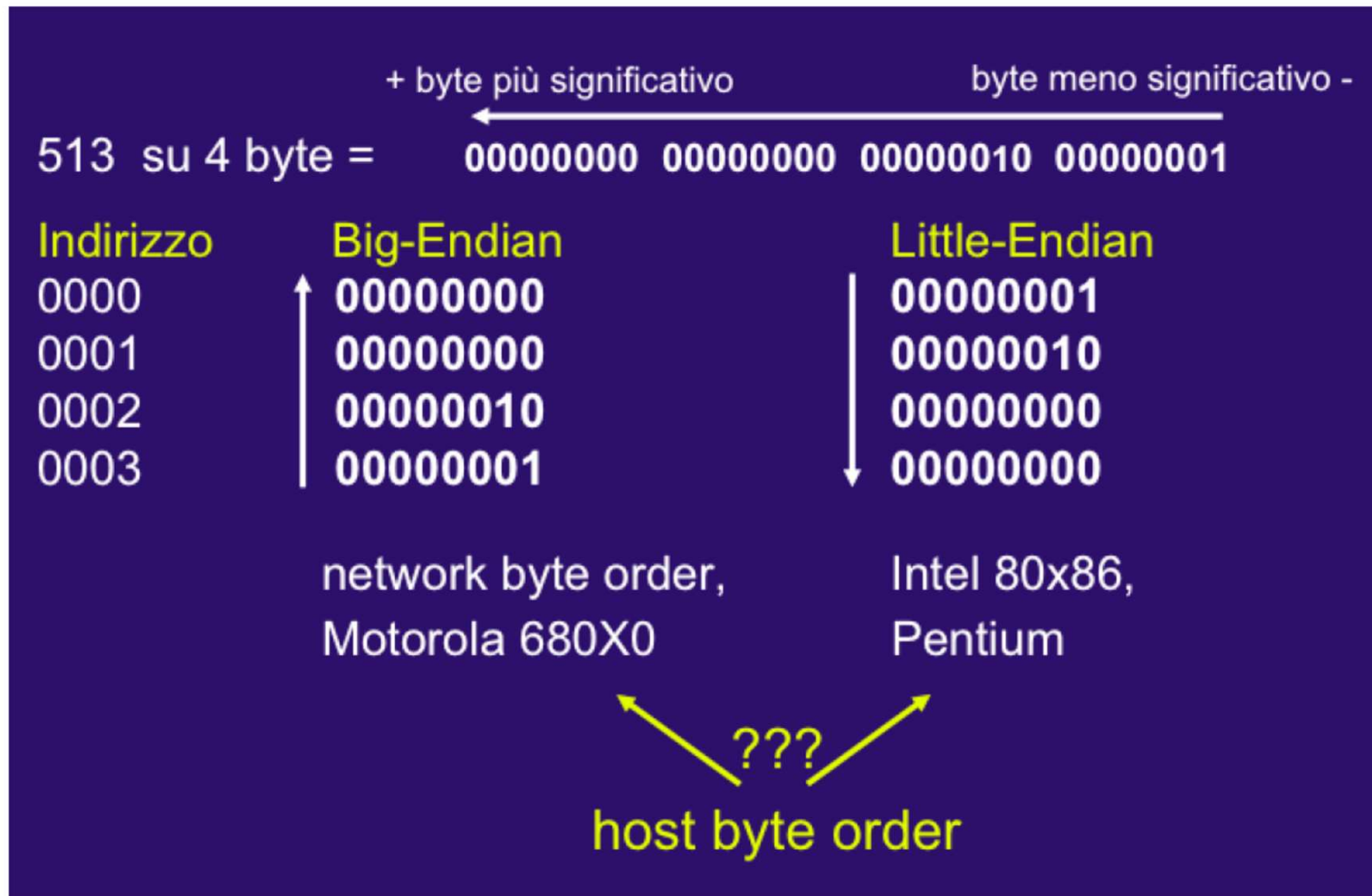
- Rappresentazione dati in memoria
 - Come viene memorizzato un intero in memoria?
 - sono usati 4 bytes
 - consideriamo 65537, cioè $2^{16} + 1$, all'indirizzo 100
 - codifica “Big Endian” (prima il byte *piu'* significativo):

00000000	00000000	10000000	00000001
100	101	102	103
 - codifica “Little Endian” (prima il byte *meno* significativo):

00000001	10000000	00000000	00000000
100	101	102	103

Socket TCP

- Rappresentazione dati in memoria



- BE: network byte order (TCP/IP), più raro come host order, es. PowerPC, SPARC
- LE: x86-64, ARM, RISC-V

Socket TCP

- Rappresentazione dati in memoria
 - i processori possono essere di entrambi i tipi
 - i protocolli TCP/IP lavorano con **big endian**
 - sono necessarie funzioni di **conversione**
 - conversioni tra unsigned:
 - #include <netinet/in.h>
 - uint32_t **htonl**(uint32_t x)
 - uint16_t **htons**(uint16_t x)
 - uint32_t **ntohl**(uint32_t x)
 - uint16_t **ntohs**(uint16_t x)

h	= Host
n	= Network (big endian)
l	= Long (4 bytes)
s	= Short (2 bytes)

Socket TCP

- Indirizzi con **notazione dotted**

```
struct sockaddr_in indirizzo;
```

```
indirizzo.sin_family = AF_INET;
```

```
if (inet_pton(AF_INET, "143.225.5.3", &indirizzo.sin_addr) != 1)
{
    perror("inet_pton");
    exit(1);
}
```

- `inet_pton` converte stringa IPv4 / IPv6 in formato binario
 - riempie direttamente `in_addr` (IPv4) o `in6_addr` (IPv6)
 - restituisce:
 - 1 = OK
 - 0 = stringa non valida
 - -1 = errore (es. famiglia non supportata)

Socket TCP

- Indirizzi con **notazione dotted**
- Esempio IPv4

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <arpa/inet.h> //header POSIX che fa parte dello stack di rete BSD-style

#define MYPORT 4242

int main(void) {
    struct sockaddr_in my_addr;
    memset(&my_addr, 0, sizeof(my_addr)); // azzera tutta la struttura

    my_addr.sin_family = AF_INET; // host byte order

    my_addr.sin_port = htons(MYPORT); // short → network byte order

    if (inet_pton(AF_INET, "10.12.110.57", &my_addr.sin_addr) != 1) {
        perror("inet_pton"); exit(1);
    }
    return 0;
}
```

Socket TCP

- Indirizzi con **notazione dotted**
- Esempio IPv6

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <arpa/inet.h>
```

```
int main(void) {
    struct sockaddr_in6 indirizzo; //indirizzo a IPv6 (a 16 byte)

    memset(&indirizzo, 0, sizeof(indirizzo)); // azzera la struttura
    indirizzo.sin6_family = AF_INET6;

    if (inet_pton(AF_INET6, "2001:db8::1234", &indirizzo.sin6_addr) != 1) {
        perror("inet_pton"); exit(1);
    }

    indirizzo.sin6_port = htons(8080); // porta

    return 0;
}
```

Socket TCP

- Indirizzi con **notazione dotted**
- IPv4 vs IPv6

```
struct sockaddr_in {  
    sa_family_t    sin_family;  
    in_port_t      sin_port;  
    struct in_addr  sin_addr;  // 4 B  
    unsigned char  sin_zero[8];  
};                          // 16 B totali
```

```
struct sockaddr_in6 {  
    sa_family_t    sin6_family;  
    in_port_t      sin6_port;  
    uint32_t       sin6_flowinfo;  
    struct in6_addr sin6_addr; // 16 B  
    uint32_t       sin6_scope_id;  
};                  // ~28-32 B totali
```

Socket TCP

- Il server chiama il bind per agganciarsi all'indirizzo
- Versione IPv4
 - INADDR_ANY per ascoltare da qualunque indirizzo: #define che vale 0.0.0.0

```
#include <netinet/in.h>
#include <arpa/inet.h>
```

```
struct sockaddr_in mio_indirizzo;
```

```
// azzera tutto
memset(&mio_indirizzo, 0, sizeof(mio_indirizzo));
```

```
mio_indirizzo.sin_family = AF_INET;
mio_indirizzo.sin_port   = htons(5200);    // network byte order
mio_indirizzo.sin_addr.s_addr = htonl(INADDR_ANY);
```

```
if (bind(fd, (struct sockaddr *)&mio_indirizzo, sizeof(mio_indirizzo)) < 0) {
    perror("bind");
    exit(1);
}
```


Socket TCP

- Il server chiama il bind per agganciarsi all'indirizzo
- Versione IPv6
 - in6addr_any per ascoltare da qualunque indirizzo (variabile globale const struct in6_addr che rappresenta l'indirizzo ::)

```
#include <netinet/in.h>
```

```
#include <arpa/inet.h>
```

```
struct sockaddr_in mio_indirizzo;
```

```
struct sockaddr_in6 mio_ind6 = {0};
```

```
mio_ind6.sin6_family = AF_INET6;
```

```
mio_ind6.sin6_port = htons(5200);
```

```
mio_ind6.sin6_addr = in6addr_any; // costante globale IPv6
```

```
bind(fd, (struct sockaddr *)&mio_ind6, sizeof(mio_ind6));
```

Socket TCP

- Il client chiama una connect
- Versione IPv4

```
#include <netinet/in.h> // sockaddr_in, htons
#include <arpa/inet.h>   // inet_pton
#include <string.h>      // memset
#include <stdio.h>       // perror
#include <stdlib.h>      // exit
```

```
/* ...creazione socket => fd ... */
```

```
struct sockaddr_in indirizzo;
memset(&indirizzo, 0, sizeof(indirizzo)); // azzera la struttura
```

```
indirizzo.sin_family = AF_INET;
indirizzo.sin_port = htons(5200);
```

```
if (inet_pton(AF_INET, "143.225.5.3", &indirizzo.sin_addr) != 1) {
    perror("inet_pton"); exit(1);}
}
```

```
if (connect(fd, (struct sockaddr *)&indirizzo, sizeof(indirizzo)) < 0) {
    perror("connect"); exit(1);
}
```

Socket TCP

- Il client chiama una connect
- Versione IPv6

```
struct sockaddr_in6 indirizzo6 = {0};
```

```
indirizzo6.sin6_family = AF_INET6;  
indirizzo6.sin6_port = htons(5200);
```

```
if (inet_pton(AF_INET6, "2001:db8::1234", &indirizzo6.sin6_addr) != 1) {  
    perror("inet_pton");  
    exit(1);  
}
```

```
connect(fd, (struct sockaddr *)&indirizzo6, sizeof(indirizzo6));
```

Socket TCP

- Il server si mette in ascolto

```
int listen(int sockfd, int lunghezza_coda);
```

- Mette il socket in modalita' di ascolto
 - in attesa di nuove connessioni
- Solo per socket SOCK_STREAM e SOCK_SEQPACKET
- Il secondo argomento specifica quante connessioni possono essere in attesa di essere accettate
 - Se il numero di connessioni in attesa supera il secondo parametro, il client riceve “connection refused”
- Restituisce 0 oppure -1

Socket TCP

- Il server accetta nuove connessioni

```
int accept(int sockfd,  
           struct sockaddr *indirizzo_client,  
           socklen_t *dimensione_indirizzo);
```

- Il secondo e terzo argomento servono ad identificare il client
 - possono essere NULL
- Restituisce un nuovo descrittore! (oppure -1)
 - crea un nuovo socket, dedicato a questa nuova connessione
 - il vecchio socket resta in ascolto
- Blocca il processo se non vi sono connessioni in attesa
 - Il socket puo' essere marcato non-blocking.

Socket TCP

- Struttura di un server

```
int fd1, fd2;
struct sockaddr_in mio_indirizzo = {0};

mio_indirizzo.sin_family    = AF_INET;
mio_indirizzo.sin_port      = htons(5200);
mio_indirizzo.sin_addr.s_addr = htonl(INADDR_ANY);

fd1 = socket(AF_INET, SOCK_STREAM, 0);

bind(fd1, (struct sockaddr *)&mio_indirizzo, sizeof(mio_indirizzo));
listen(fd1, 5);

fd2 = accept(fd1, NULL, NULL);
/* ... */
close(fd2);
close(fd1);
```

Socket TCP

- Struttura di un server

```
#include <netinet/in.h>
#include <arpa/inet.h>
#include <string.h>
#include <unistd.h>    // close

int main(void) {
    int s, c;
    struct sockaddr_in addr = {0};

    addr.sin_family    = AF_INET;
    addr.sin_port      = htons(5200);
    addr.sin_addr.s_addr = htonl(INADDR_ANY);

    s = socket(AF_INET, SOCK_STREAM, 0);
    bind(s, (struct sockaddr *)&addr, sizeof(addr));
    listen(s, 5);
    c = accept(s, NULL, NULL);
    /* ... comunico con c ... */
    close(c); close(s);
}
```

Socket TCP

- Struttura di un client

```
#include <netinet/in.h>
#include <arpa/inet.h>
#include <string.h>
#include <unistd.h>    // close

int main(void) {
    int s;
    struct sockaddr_in addr = {0};

    addr.sin_family = AF_INET;
    addr.sin_port   = htons(5200);

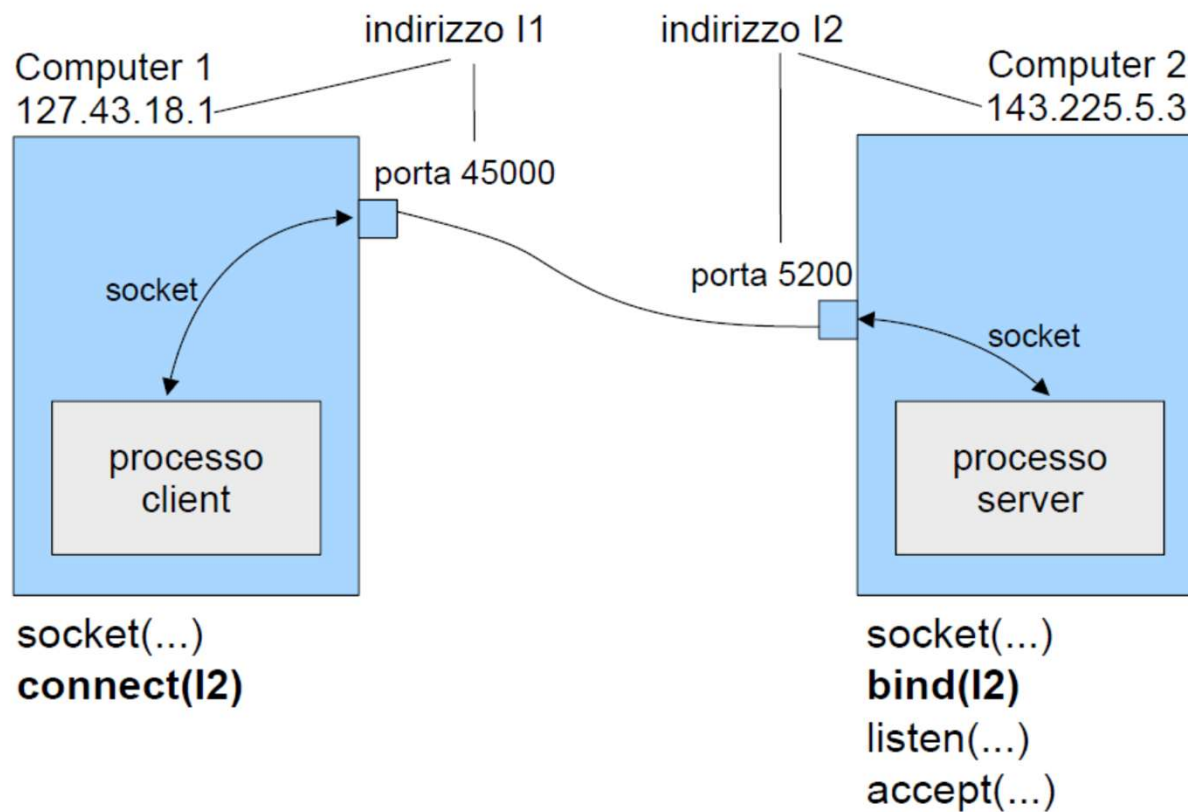
    inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);

    s = socket(AF_INET, SOCK_STREAM, 0);
    connect(s, (struct sockaddr *)&addr, sizeof(addr));

    /* ... comunico con s ... */
    close(s);
}
```


Socket TCP

- Schema di connessione



Socket TCP

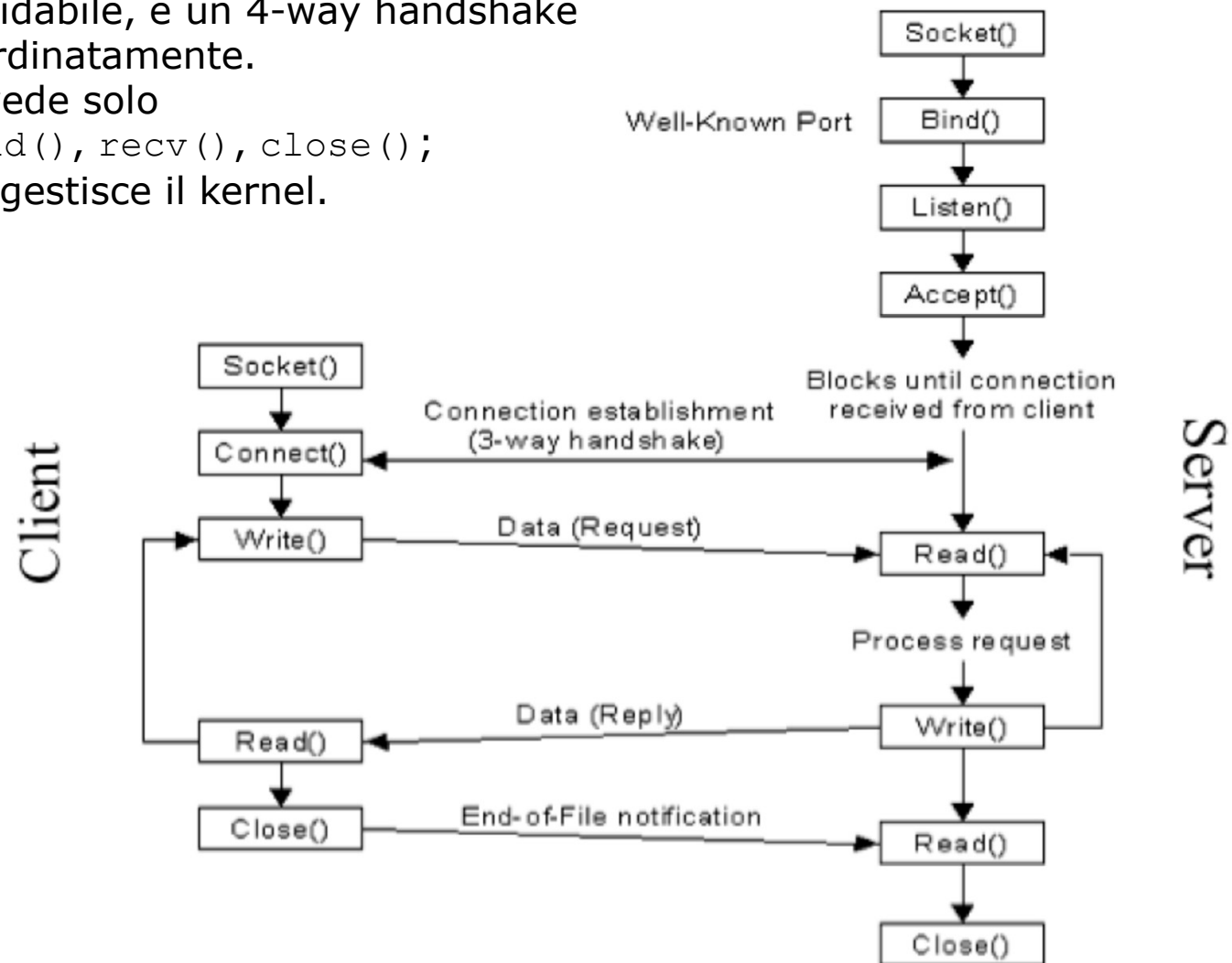
- Schema di connessione

TCP ha un 3-way handshake per stabilire una connessione affidabile, e un 4-way handshake per chiuderla ordinatamente.

L'applicazione vede solo

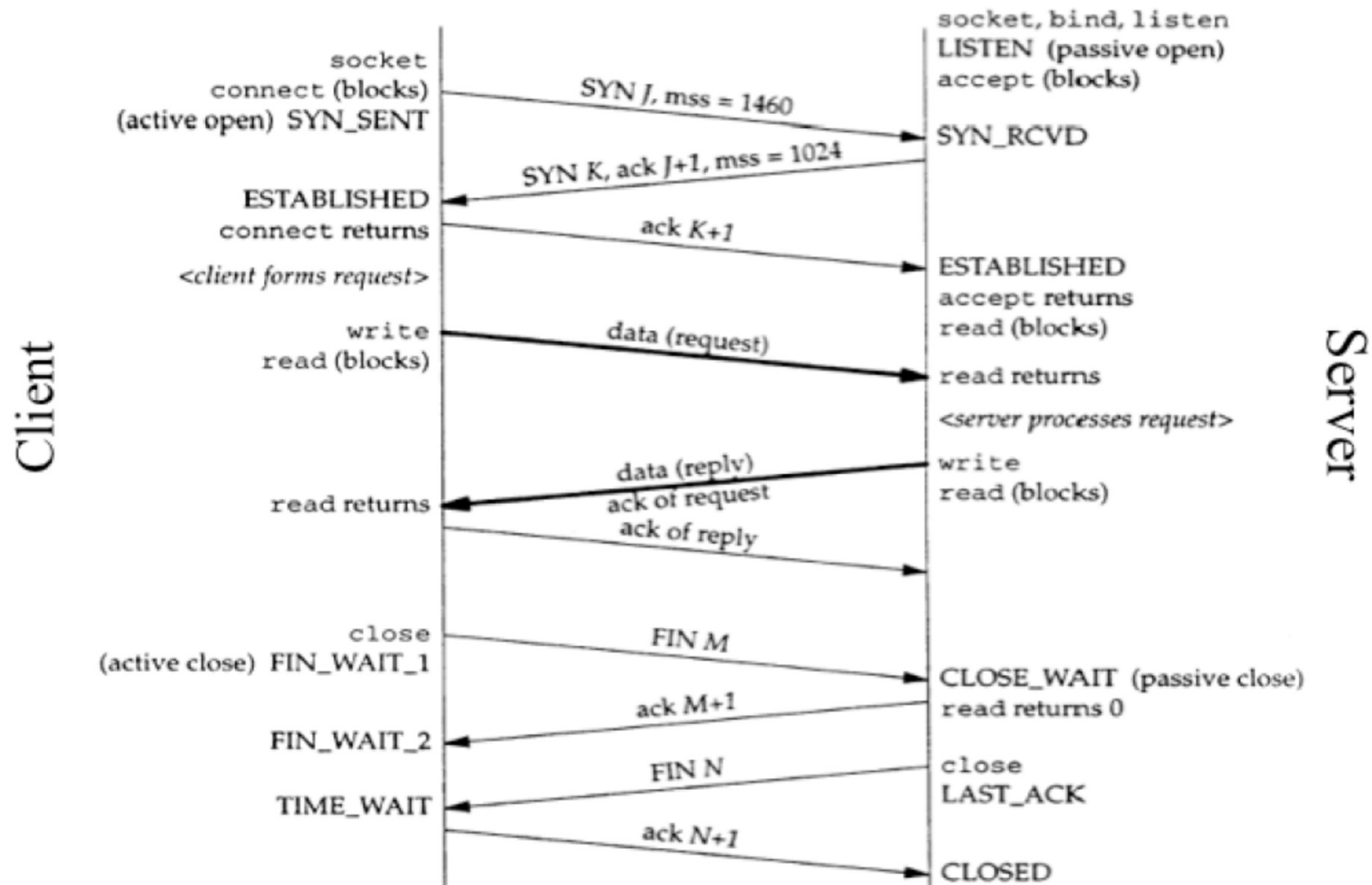
`connect()`, `send()`, `recv()`, `close()`;

tutto il resto lo gestisce il kernel.



Socket TCP

- Schema di connessione
 - handshake segmenti TCP



Socket TCP

- Scrittura con write

```
#include <unistd.h>
ssize_t write(int fd,
              const void *buf,
              size_t count);
```

- invia il contenuto del buffer buf al socket specificato
- si usa esclusivamente con SOCK_STREAM
- restituisce il numero di byte inviati oppure -1 in caso di errore
- è la stessa funzione che consente la scrittura su file

Socket TCP

- Ricezione con read

```
#include <unistd.h>
ssize_t read(int fd,
             void *buf,
             size_t count);
```

- solo per socket connessi (SOCK_STREAM)
- legge un messaggio di lunghezza massima len dal socket
- se non c'è alcun messaggio, il programma rimane sospeso
 - chiamata bloccante
- la funzione ritorna il numero di byte letti, -1 in caso di errore
- è la stessa funzione che consente la lettura da un file

Socket TCP

- Lettura scrittura

Stessa interfaccia per scrittura su file:

- Si possono passare le sd a processi figli che possono non distinguere tra sd e fd
- Possono essere usati anche con processi che lavorano su file locali.
- Esistono altre funzioni specifiche per leggere e scrivere su socket: 3 modalità di send, 3 modalità di receive.

Socket TCP

- Lettura scrittura

- **Send & Recv:**

- **send** è come write (richiede connessione stabilita), ma ha flag per specificare modalità di scrittura,

- **sendto** permette la scrittura su connectionless socket,

- **sendmsg** per specificare buffer multipli

- **recv** come read con flag,

- **recvfrom** per ottenere indirizzo della fonte,

- **rcvmsg** per ricevere da buffer multipli

Socket TCP

- Lettura scrittura

```
#include <sys/types.h>
#include <sys/socket.h>
int send(int s,
        const void *msg, int len,
        unsigned int flags);
int sendto(int s, const void *msg, int len,
           unsigned int flags,
           const struct sockaddr *to, int tolen);
```

man 2 send

- `send()` può essere utilizzata solo se `s` è stato connesso; `sendto()` sempre perché richiede di specificare l'indirizzo di destinazione
- restituisce `-1` in caso di errore oppure il numero di byte effettivamente trasmessi
- `flags` si può lasciare a zero

Socket TCP

- Lettura scrittura

```
#include <sys/types.h>
#include <sys/socket.h>
int recv(int s,
         void *buf, int len,
         unsigned int flags);
int recvfrom(int s, void *buf, int len,
             unsigned int flags,
             struct sockaddr *from,
             int *fromlen);
```

man 2 recv

- ricevono in `buf` non più di `len` byte. Se `from` non è `NULL`, la struttura `sockaddr` verrà riempita con l'indirizzo del mittente
- restituisce `-1` in caso di errore oppure il numero di byte effettivamente ricevuti
- `flags` si può lasciare a zero

Socket TCP

- Lettura scrittura

```
#include <sys/types.h>
#include <sys/socket.h>
int recvfrom(int s, void *buf, int len,
             unsigned int flags,
             struct sockaddr *from,
             int *fromlen);
```

man 2 recv

- ricevono in `buf` non più di `len` byte. Se `from` non è `NULL`, la struttura `sockaddr` verrà riempita con l'indirizzo del mittente
- restituisce `-1` in caso di errore oppure il numero di byte effettivamente ricevuti
- se non arriva alcun messaggio, il programma rimane sospeso (la chiamata è "bloccante")
- `flags` si può lasciare a zero

Socket TCP

■ Send/Receive

- accettano un parametro “flags” che permette comportamenti extra rispetto a write/read:

```
ssize_t send(int sock, const void *buf, size_t len, int flags);  
ssize_t recv(int sock, void *buf, size_t len, int flags);
```

flag

MSG_DONTWAIT

MSG_PEEK

MSG_WAITALL

effetto

non blocca

legge senza consumare receive queue

aspetta finché il len non è raggiunto

- MSG_DONTWAIT su recv() non aspetta il dato (es. polling)
- MSG_DONTWAIT su send() non aspetta se il buffer è pieno
- MSG_PEEK su recv() per testare un dato prima di rimuoverlo
- MSG_WAITALL su recv() aspetta che il len sia raggiunto
- MSG_NOSIGNAL su send() evita il SIGPIPE
- Senza flag stesso comportamento di read/write

Socket TCP

■ Send/Receive

- accettano un parametro “flags” che permette comportamenti extra rispetto a write/read:

```
ssize_t send(int sock, const void *buf, size_t len, int flags);  
ssize_t recv(int sock, void *buf, size_t len, int flags);
```

flag

MSG_DONTWAIT

MSG_PEEK

MSG_WAITALL

effetto

non blocca

legge senza consumare receive queue

aspetta finché il len non è raggiunto

- MSG_DONTWAIT su recv() non aspetta il dato (es. polling)
- MSG_DONTWAIT su send() non aspetta se il buffer è pieno
- MSG_PEEK su recv() per testare un dato prima di rimuoverlo
- MSG_WAITALL su recv() aspetta che il len sia raggiunto
- MSG_NOSIGNAL su send() evita il SIGPIPE
- Senza flag stesso comportamento di read/write

Socket UDP

■ SendTo/ReceiveFrom

- Server UDP minimale

```
#include <netinet/in.h>
#include <arpa/inet.h>
#include <string.h>
#include <unistd.h>
```

```
int main(void) {
    int s;
    struct sockaddr_in addr = {0};
    char buf[1024];

    s = socket(AF_INET, SOCK_DGRAM, 0);

    addr.sin_family    = AF_INET;
    addr.sin_port      = htons(5200);
    addr.sin_addr.s_addr = htonl(INADDR_ANY);

    bind(s, (struct sockaddr *)&addr, sizeof(addr));
```

Socket UDP

- **SendTo/ReceiveFrom**

- Server UDP minimale

```
while (1) {  
    struct sockaddr_in from = {0};  
    socklen_t flen = sizeof(from);  
  
    int n = recvfrom(s, buf, sizeof(buf), 0,  
                    (struct sockaddr *)&from, &flen);  
  
    sendto(s, buf, n, 0,  
           (struct sockaddr *)&from, flen);  
}  
  
close(s);  
}
```

Socket UDP

- **SendTo/ReceiveFrom**

- Client UDP minimale

```
#include <netinet/in.h>
#include <arpa/inet.h>
#include <string.h>
#include <unistd.h>
#include <stdio.h>
```

```
int main(void) {
    int s;
    struct sockaddr_in addr = {0};
    char buf[1024] = "ciao";

    s = socket(AF_INET, SOCK_DGRAM, 0);

    addr.sin_family = AF_INET;
    addr.sin_port = htons(5200);
    inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);
```

Socket UDP

- **SendTo/ReceiveFrom**

- Client UDP minimale

```
sendto(s, buf, strlen(buf), 0,  
      (struct sockaddr *)&addr, sizeof(addr));
```

```
int n = recvfrom(s, buf, sizeof(buf), 0, NULL, NULL);
```

```
/* ... usa buf ... */  
buf[n] = '\0';  
printf("ricevuto dal server: %s\n", buf);
```

```
close(s);  
}
```


Socket TCP

■ Esempio

- Server incrementa un numero passato dal client

```
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <signal.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>
```

```
static int s = -1; // listening socket
```

```
void handler(int sig) {
    (void)sig;
    if (s >= 0) {
        shutdown(s, SHUT_RDWR); // forza accept a ritornare
    }
}
```

Socket TCP

■ Esempio

- Server incrementa un numero passato dal client

```
int main(void) {  
  
    signal(SIGINT, handler);  
    s = socket(AF_INET, SOCK_STREAM, 0);  
  
    struct sockaddr_in addr = {0};  
    addr.sin_family      = AF_INET;  
    addr.sin_port        = htons(5201);  
    addr.sin_addr.s_addr = htonl(INADDR_ANY);  
    bind(s, (struct sockaddr*)&addr, sizeof(addr));  
    listen(s, 1);  
  
    printf("[server] in ascolto (Ctrl-C per uscire)\n");  
  
    while (1) {  
        int c = accept(s, NULL, NULL);  
        if (c < 0) {  
            // probabile uscita causata da shutdown()  
            break;  
        }  
    }
```

Socket TCP

■ Esempio

- Server incrementa un numero passato dal client

```
uint32_t net_x;
ssize_t r = recv(c, &net_x, sizeof(net_x), 0);

if (r == sizeof(net_x)) {
    uint32_t x = ntohl(net_x);
    uint32_t next = x + 1;
    uint32_t net_next = htonl(next);
    send(c, &net_next, sizeof(net_next), 0);
}

close(c);
}

close(s);
return 0;
}
```

Socket TCP

■ Esempio

- Client passa un numero

```
int main(void) {
```

```
    int s = socket(AF_INET, SOCK_STREAM, 0);
```

```
    struct sockaddr_in srv = {0};
```

```
    srv.sin_family = AF_INET;
```

```
    srv.sin_port = htons(5201);
```

```
    inet_pton(AF_INET, "127.0.0.1", &srv.sin_addr);
```

```
    connect(s, (struct sockaddr*)&srv, sizeof(srv));
```

```
    int x;
```

```
    printf("Inserisci un numero: ");
```

```
    if (scanf("%d", &x) != 1) {
```

```
        printf("Input non valido.\n");
```

```
        close(s);
```

```
        return 1;
```

```
    }
```

Socket TCP

■ Esempio

- Client passa un numero

```
uint32_t net_x = htonl((uint32_t)x);
send(s, &net_x, sizeof(net_x), 0);

uint32_t net_res;
ssize_t r = recv(s, &net_res, sizeof(net_res), 0);

if (r == sizeof(net_res)) {
    uint32_t res = ntohl(net_res);
    printf("Risposta dal server: %u\n", res);
} else {
    printf("Errore durante recv\n");
}

close(s);
return 0;
}
```

Socket TCP

- **Esercizio**

- Modificare client e server in modo tale che il client invii due numeri e riceva la somma dal server
- Il client cicla fino ad interruzione